

ARTI 403 - Image Processing

Lab 1 Report

Basics of Programming with Python - Image Loading, Visualization, Storage, and Array Operations

Student Name	Abdullah Hejji Alnuwaysir
Student ID	2240005501
Section	7MA2
Course	ARTI 403 - Image Processing
Lab	1
Date	01 October 2026

Outcome #1: explain how digital images are represented and manipulated in a computer.

Contents

1	Lab Overview	1
2	Task 1 - Python Basics	1
3	Task 2 - Loading and Visualizing Images	1
4	Task 3 - Image Storing	2
5	Task 4 - Displaying an Image as an Array	2
6	Assessment - NumPy Array Operations	2
7	Conclusion	5
8	Submission Contents	5

1. Lab Overview

This report documents the completed activities for Lab 1, which introduces Python programming for image processing. The work was done in a Jupyter Notebook using **OpenCV**, **PIL**, **NumPy** and **Matplotlib**. The lab covers loading, displaying and saving images, representing an image as an array, and basic NumPy operations on the image array.

File	Type	Used in
cameraman.tif	Grayscale, 512 x 512	Loading with OpenCV, saving, array display
lena_gray_256.tif	Grayscale, 256 x 256	Loading with PIL, saving, array operations

2. Task 1 - Python Basics

A warm-up exercise with basic Python arithmetic and NumPy arrays (**a = 10**, **b = 3**, and an array of the numbers 1 to 10).

```
a + b = 13
a / b = 3.3333333333333335
a // b = 3
a % b = 1

Array: [ 1  2  3  4  5  6  7  8  9 10]
Sum: 55
Mean: 5.5
Reshaped (2x5):
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

Observation: `//` performs integer (floor) division and `%` returns the remainder, while **reshape(2, 5)** rearranges the 10 values into 2 rows and 5 columns without changing them.

3. Task 2 - Loading and Visualizing Images

Images were loaded with two libraries and displayed with Matplotlib: **cv2.imread()** for the Cameraman image and **Image.open()** from PIL for the second image.



Figure 1. Loaded with OpenCV (BGR).



Figure 2. Loaded with PIL (grayscale).

Observation: OpenCV loads images in BGR channel order, so a color image would look wrong if displayed directly with Matplotlib (which expects RGB). Both test images are grayscale, so the channel order is not visible here. PIL loads the image as a single-channel object that is displayed with a gray colormap.

4. Task 3 - Image Storing

Each loaded image was written back to disk, once with each library.

Library	Code	Output file
OpenCV	<code>cv2.imwrite('new_image.jpg', img)</code>	new_image.jpg
PIL	<code>img2.save('new_image2.jpg')</code>	new_image2.jpg

Observation: Both libraries choose the file format from the extension, so saving to **.jpg** converts the original TIFF images to JPEG.

5. Task 4 - Displaying an Image as an Array

A digital image is stored as an array of numbers. The arrays of the two loaded images were printed (only the corners are shown by NumPy).

Source	Array shape	Meaning
OpenCV (cameraman.tif)	(512, 512, 3)	512 rows x 512 columns x 3 channels (B, G, R)
PIL to NumPy (lena_gray_256.tif)	(256, 256)	256 rows x 256 columns, single gray channel

OpenCV array (first values of the top row):	PIL array:
<code>[[[200 200 200]</code>	<code>[[143 86 113 ... 121 121 118]</code>
<code>[200 200 200]</code>	<code>[200 179 185 ... 119 120 118]</code>
<code>[200 200 200]</code>	<code>[227 218 213 ... 120 120 119]</code>
<code>...</code>	<code>...</code>
<code>[190 190 190]]</code>	<code>[172 166 148 ... 84 21 1]]</code>

Observation: Each element is a pixel intensity from 0 (black) to 255 (white). OpenCV returns three channels even for a grayscale file, and the three values of every pixel are identical (for example 200, 200, 200), while PIL gives a two-dimensional array with one value per pixel.

6. Assessment - NumPy Array Operations

The following operations were applied to **img_array**, the grayscale image (lena_gray_256.tif) loaded with PIL and converted to a NumPy array.

6.1 Basic Inspection

Property	Code	Value
Shape	<code>img_array.shape</code>	(256, 256)
Data type	<code>img_array.dtype</code>	uint8
Total pixels	<code>img_array.size</code>	65536
Dimensions	<code>img_array.ndim</code>	2

6.2 Indexing and Slicing



The pixel at row 100, column 100 was read and the top-left 100 x 100 region was cropped with slicing.

```
img_array[100, 100] -> 9
img_array[0:100, 0:100] -> shape (100, 100)
```

Observation: indexing returns a single intensity value (9, a very dark pixel), and slicing returns a sub-array that is displayed as a smaller image.

Figure 3. Cropped 100 x 100 region.

6.3 Flipping and Rotating

np.fliplr, **np.flipud** and **np.rot90** were applied to the array.



Figure 4. Horizontal flip, vertical flip and 90-degree rotation.

Observation: The horizontal flip mirrors the image left to right, the vertical flip mirrors it top to bottom, and **np.rot90** rotates it by 90 degrees counterclockwise. These operations only rearrange the array, so no pixel values change.

6.4 Arithmetic Operations (Brightness Adjustment)

A constant of 60 was added to or subtracted from every pixel. The array was first converted to **int** and the result was clipped to [0, 255] with **np.clip** before converting back to uint8.

```
brighter = np.clip(img_array.astype(int) + 60, 0, 255).astype(np.uint8)
darker   = np.clip(img_array.astype(int) - 60, 0, 255).astype(np.uint8)
```

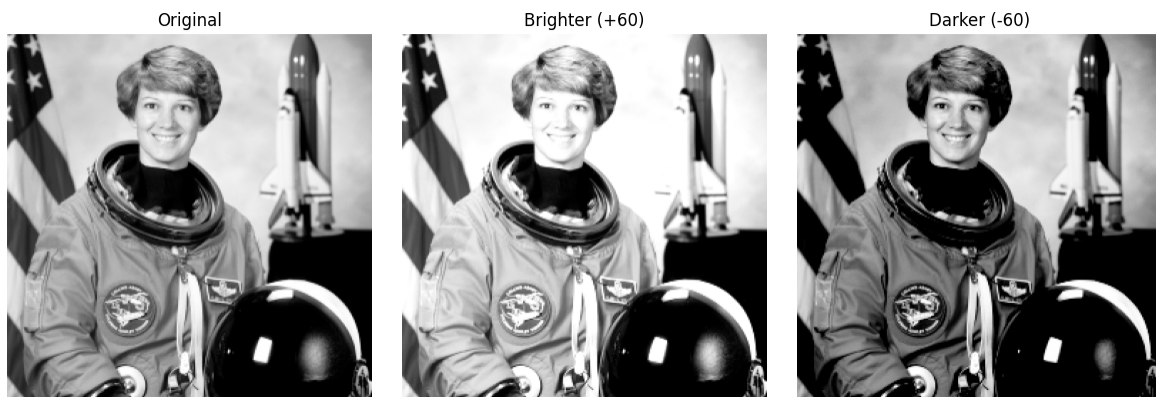


Figure 5. Original, brighter (+60) and darker (-60) images.

Observation: Adding 60 brightens the image and subtracting 60 darkens it. Clipping prevents uint8 overflow, so values above 255 stay at 255 and values below 0 stay at 0 instead of wrapping around.

6.5 Thresholding (Binary Image)



Figure 6. Binary image (threshold = 128).

Pixels brighter than the threshold of 128 were set to 255 (white) and all other pixels to 0 (black).

```
binary = np.where(img_array > 128, 255, 0)
```

Observation: the result contains only two values, so smooth gray transitions are lost and only the main bright and dark regions remain.

6.6 Basic Statistics

Statistic	Code	Value
Minimum pixel value	<code>img_array.min()</code>	0
Maximum pixel value	<code>img_array.max()</code>	255
Mean pixel value	<code>img_array.mean()</code>	115.52
Standard deviation	<code>img_array.std()</code>	74.49

Observation: The image uses the full intensity range (0 to 255). A mean of about 115.5 shows it is slightly darker than mid-gray, and the large standard deviation (about 74.5) indicates high contrast.

7. Conclusion

- Configured the Python image-processing environment (Anaconda, Jupyter, OpenCV, PIL, NumPy, Matplotlib).
- Loaded and visualized images using both OpenCV and PIL.
- Saved images to disk using both libraries.
- Represented an image as a NumPy array and inspected its shape and values.
- Performed NumPy operations on the image array: slicing/cropping, flipping, rotating, brightness adjustment, thresholding, and basic statistics.

8. Submission Contents

The submission contains the completed Jupyter notebook

Lab1_ARTI403_Image_Processing.ipynb together with the images used by it (cameraman.tif and lena_gray_256.tif).

Student: Abdullah Hejji Alnuwaysir | **ID:** 2240005501 | **Section:** 7MA2